

TEXT EDITOR UNDO/REDO SYSTEM

เอกสารโครงร่างสไลด์และเนื้อหาการนำเสนอเชิงลึก - กลุ่มที่ 3 (Snapshot vs Command Method)

กำหนดการและลำดับขั้นตอนการนำเสนอ

- **ส่วนที่ 1:** ทฤษฎี, การวิเคราะห์ปัญหา, อัลกอริทึมทั้ง 2 แบบ และ Logic การทำ Inverse Operation
- **ส่วนที่ 2:** การอธิบายความถูกต้อง (Correctness Proof) และการวิเคราะห์ Complexity (Big-O Analysis)
- **ส่วนที่ 3:** Live Demo สาธิตโปรแกรมภาษา Java ตาม Test Cases บังคับและ Edge Cases
- **ส่วนที่ 4:** ผลการทดลองเปรียบเทียบประสิทธิภาพ (Memory & Execution Time Graph)
- **ส่วนที่ 5:** สรุปอภิปรายผล, Generative AI Reflection และ Q&A

สไลด์ที่ 1: ข้อมูลกลุ่มและหัวข้อโครงงาน

- **หัวข้อ:** การศึกษาวิเคราะห์และเปรียบเทียบประสิทธิภาพระบบ Undo/Redo ใน Text Editor ระหว่าง Snapshot Method และ Command (Delta) Method
- **กลุ่มที่ 3:** [ใส่ชื่อสมาชิกในกลุ่มทุกคนพร้อมรหัสนักศึกษา]
- **รายวิชา:** การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack (ภาษา Java)

สไลด์ที่ 2: Case Story & Problem Analysis

- **ปัญหาหลัก:** ออกแบบระบบ Undo/Redo สำหรับ Text Editor ที่รองรับคำสั่ง INSERT, DELETE, และ REPLACE
- **เงื่อนไขสำคัญ:** เมื่อมีการ Undo แล้วผู้ใช้ทำ Action ใหม่ ประวัติใน Redo Stack ทั้งหมดต้องถูกล้างทันที (Redo Stack Clearing Rule)
- **วิเคราะห์ Input/Output & Constraints:**
 - **Input:** คำสั่งแก้ไขข้อความ และโครงสร้างข้อมูล Action (Action ID, Action Type, Position, Old Text, New Text, Timestamp)
 - **Output:** ข้อความหลังประมวลผล และการแสดงสถานะ Stack
 - **Constraints & Edge Cases:** Index หลุดขอบเขต, ข้อความว่าง, การเรียก Undo/Redo เมื่อ Stack ว่าง

สไลด์ที่ 3: Algorithm A – Snapshot Method

- **แนวคิดหลัก:** ทุกครั้งที่มี Action เกิดขึ้น ระบบจะบันทึกข้อความทั้งฉบับ (Full Document State) ลงใน Undo Stack
- **ขั้นตอนการทำงาน:**
 - **Action ใหม่:** Push ข้อความปัจจุบันลง undoStack และ Clear redoStack
 - **Undo:** Pop สถานะข้อความเดิมจาก undoStack นำไปใส่ redoStack แล้วปรับข้อความปัจจุบันให้เท่ากับค่าที่ Pop ออกมา
 - **Redo:** Pop สถานะจาก redoStack กลับเข้า undoStack แล้วอัปเดตข้อความปัจจุบัน
- **ข้อดี/ข้อจำกัด:** พัฒนาง่าย แต่สิ้นเปลืองหน่วยความจำมหาศาลเนื่องจากต้อง Copy ข้อความทั้งหมดทุกครั้ง ($O(n)$ ต่อหนึ่ง Action)

สไลด์ที่ 4: Algorithm B – Command / Delta Method & Inverse Operations

- **แนวคิดหลัก:** เก็บเฉพาะข้อมูลส่วนต่างของการเปลี่ยนแปลง (Action/Command Delta) และใช้ Inverse Operation ในการย้อนกลับ
- **Logic ของ Inverse Operation:**
 - **INSERT:** Inverse คือการลบข้อความ (Delete) ความยาวเท่ากับ newText ออกจากตำแหน่ง position
 - **DELETE:** Inverse คือการแทรกข้อความ (Insert) นำ oldText กลับเข้าตำแหน่ง position
 - **REPLACE:** Inverse คือการทำ Replace ซ้ำ โดยนำ oldText ใส่กลับเข้าไปแทนที่ newText ที่ตำแหน่ง position

สไลด์ที่ 5: Pseudocode Deep-Dive (Algorithm B)

ALGORITHM ExecuteUndoCommand(document, command)

INPUT: document คือ ข้อความปัจจุบัน, command คือ Action บน Top ของ UndoStack

OUTPUT: document ที่คืนค่าเรียบร้อยแล้ว

1. SWITCH command.actionType DO
2. CASE INSERT:
3. Delete command.newText.length characters at command.position from document
4. CASE DELETE:
5. Insert command.oldText at command.position into document
6. CASE REPLACE:
7. Replace command.newText with command.oldText at command.position in document
8. END SWITCH
9. RETURN document

สไลด์ที่ 6: Step-by-Step State Demonstration

- **ตัวอย่างสถานการณ์:** ข้อความเริ่มต้น: "Hello"
- **Step 1:** INSERT " World" at Pos 5 → Document: "Hello World" | undoStack: [INSERT(5, "", " World")]
- **Step 2:** DELETE "Hello " at Pos 0 → Document: "World" | undoStack: [INSERT, DELETE(0, "Hello ", "")]
- **Step 3:** UNDO (Reverse DELETE) → Insert "Hello " กลับที่ Pos 0 → Document: "Hello World"
- **Step 4:** INSERT "!" at Pos 11 → Redo Stack ถูกล้างทันที

สไลด์ที่ 7: Correctness Argument (ความถูกต้องของอัลกอริทึม)

- **Precondition:** เอกสารอยู่ในสถานะ S_0 และ Stack ทั้งสองว่างเปล่า
- **Loop/Operation Invariant:** สำหรับ Action ใดๆ A_i ที่แปลงสถานะเอกสารจาก $S_{i-1} \rightarrow S_i$ การประยุกต์ใช้ Inverse Operation A_{i-1} จะได้ผลลัพธ์เป็น S_{i-1} เสมอ โดยไม่ทำให้ข้อมูลส่วนอื่นสูญหายหรือเปลี่ยนแปลง
- **Postcondition:** การทำ Undo k ครั้ง ตามด้วย Redo k ครั้ง จะคืนค่าข้อความสู่สถานะล่าสุดเสมอโดยตรง
- **Termination:** ทุก Operation ทำงานกับขอบเขตข้อมูลที่มีขนาดแน่นอน (m ตัวอักษร) จึงไม่มี infinite loop และจบการทำงานได้เสมอ

สไลด์ที่ 8: Theoretical Big-O Analysis

Operation / Metric	Snapshot Method (Alg A)	Command Method (Alg B)
Undo / Redo Time Complexity	$O(n)$ (เนื่องจากการ Copy สตริงยาว n)	$O(m)$ (m คือความยาวของข้อความที่เปลี่ยนไป)
Space Complexity per Action	$O(n)$ (n คือความยาวเอกสารทั้งหมด)	$O(m)$ (m คือความยาวเฉพาะส่วน Delta)
Total Auxiliary Space (k Actions)	$O(k \times n)$	$O(k \times m)$

สไลด์ที่ 9: Architecture & Java Implementation

- โครงสร้างคลาส (Class Structure):
 - Main: จัดการ เมนูรับคำสั่ง และแสดงสถานะ
 - TextDocument: เก็บข้อความโดยใช้ StringBuilder เพื่อประสิทธิภาพสูงสุด
 - ActionCommand: Data Model สำหรับเก็บโครงสร้างข้อมูล Delta
 - SnapshotManager & CommandManager: คลาสจัดการ Stack และอัลกอริทึมทั้งสองวิธี
- โครงสร้าง Stack: ใช้ `Deque<T> stack = new ArrayDeque<>()` ตามข้อแนะนำ

สไลด์ที่ 10: Performance Experiment Setup & Results

- **พารามิเตอร์การทดสอบ:** ความยาวข้อความ (n): 100, 1,000, 10,000, 100,000 ตัวอักษร และจำนวน Action (k): 100, 1,000, 10,000 รายการ
- **Memory Usage:** Command Method ใช้พื้นที่น้อยกว่า Snapshot Method ถึง 95-99% ในไฟล์ขนาดใหญ่
- **Execution Time:** Command Method มีเวลาประมวลผลสม่ำเสมอ ในขณะที่ Snapshot Method ใช้เวลาเพิ่มขึ้นแบบเส้นตรงตามความยาวข้อความ n

สไลด์ที่ 11: Analytical Questions Breakdown (เจาะลึกคำถามวิเคราะห์)

- **1. Snapshot ใช้พื้นที่เท่าใดเมื่อมี m Action และข้อความยาว n ?**
ตอบ: $O(m \times n)$ เพราะทุก Action บันทึกสตริงยาว n ทั้งหมด
- **2. String และ StringBuilder ส่งผลต่อ Complexity อย่างไร?**
ตอบ: String เป็น Immutable การแก้ไขหรือย้ายข้อมูลต้อง Copy ใหม่เสมอทำให้เกิด overhead $O(n)$ ส่วน StringBuilder รองรับ Mutable operations แบบ $O(m)$ จึงเร็วกว่ามาก
- **3. Snapshot และ Command เหมาะกับระบบประเภทใด?**
ตอบ: Snapshot เหมาะกับระบบขนาดเล็กที่เน้นความง่ายในการพัฒนา หรือ State ย่อยซับซ้อน ส่วน Command เหมาะกับระบบ Text Editor ระดับใช้งานจริงที่ต้องรองรับไฟล์ขนาดใหญ่

สไลด์ที่ 12: Generative AI Reflection & Conclusion

- **การใช้งาน AI:** ใช้ AI ช่วยร่าง Pseudocode และสร้างโครง Test Cases
- **การตรวจสอบ:** ตรวจพบข้อผิดพลาดที่ AI ไม่ได้ล้าง redoStack เมื่อมี New Action เกิดขึ้น จึงทำการแก้ไข Code ให้ถูกต้องตามโจทย์

ข้อสรุปของกลุ่ม: ทางกลุ่มเลือก **Algorithm B (Command Method)** เป็นแนวทางที่เหมาะสมที่สุดสำหรับ Text Editor เนื่องจากประหยัดหน่วยความจำมหาศาลและมี Time Complexity สำหรับ Undo/Redo ที่ต่ำกว่าอย่างเห็นได้ชัด